

Chapter 8

Simulation of Queues and Related Models

Lectures on Monte Carlo Theory

Leskelä & Vihola

Chapter Overview

- 1 8.1 Preliminaries
- 2 8.2 Modeling Arrival Streams
 - 8.2.1 Homogeneous Poisson Process
 - 8.2.2 Non-homogeneous Poisson Process
 - 8.2.3 Poisson Process on General Spaces
- 3 8.3 Birth and Death Processes / Markovian Queues
- 4 8.4 Basic Concepts of Queueing Theory
- 5 8.5 Discrete-Event Simulations
 - 8.5.1 GI/G/c Queue Simulation
- 6 8.6 Slotted ALOHA
- 7 8.7 Ergodically Stable Processes
 - 8.7.1 Formula for Asymptotic Variance (TAVC)
 - 8.7.2 TAVC for Other Systems
- 8 8.8 Regenerative Processes
- 9 8.9 Error Analysis of M/M/1 and M/G/ ∞
- 10 8.10 Summary

8.1 Preliminaries – What Is a Queueing System? I

Simulation methods are indispensable for analyzing complex stochastic systems. While mathematical models provide powerful abstractions for queueing networks or telecommunication systems, closed-form solutions are often unavailable due to real-world complexity. Simulation therefore becomes the critical tool that enables practitioners to study system behavior under various configurations and constraints. This chapter focuses on the simulation of queueing models and related systems, offering both theoretical insights and practical implementations. Queueing systems appear in a vast range of real-world applications: call center optimization, inventory management, manufacturing lines, and telecommunications. By simulating these systems we can evaluate performance characteristics such as average waiting times, system utilization, and throughput, even in cases where analytical solutions are completely intractable.

8.1 Preliminaries – What Is a Queueing System? II

The General Model: Tasks, Arrivals, and Service

A “system” is characterized by two ingredients:

- **Tasks** (also called jobs, customers, packets, or calls) that arrive at the system according to some random mechanism.
- A **processing stage** in which tasks are served and eventually leave the system.

The quantity of interest is typically a *performance characteristic* such as the number of tasks currently waiting, the average time a task spends in the system, or the fraction of time a server is busy.

8.1 Preliminaries – What Is a Queueing System? III

Two ways to track performance. Performance characteristics can be observed in two complementary ways, depending on whether we index by customer number or by continuous time.

First, one may observe a *discrete-time sequence*: for example, the waiting time W_i (W sub i) of the i -th arriving task gives a sequence $W_1, W_2, \dots$ indexed by customer number. Each W_i is the time that customer number i must wait in the queue before service begins.

Second, one may observe a *continuous-time process*: for example, the total number of tasks $L(t)$ (L of t) present in the system at time t is a right-continuous step process that jumps up by 1 at each arrival and down by 1 at each departure.

8.1 Preliminaries – What Is a Queueing System? IV

Time Average vs. Ensemble Average

Two fundamentally different averages arise in simulation:

- The **time average** of a process $X(t)$ over the interval $[0, t]$:

$$\hat{X}(t) = \frac{1}{t} \int_0^t X(s) ds.$$

Here $X(s)$ is the process value at time s , and $\hat{X}(t)$ (X -hat of t) is its running average up to time t . This is what a *single long simulation run* produces.

- The **ensemble average** at time t is $\mathbb{E}[X(t)]$ (E of X of t), the expected value computed by averaging over all possible random trajectories.

The central question of simulation output analysis is: *when does $\hat{X}(t)$ converge to a meaningful steady-state quantity as $t \rightarrow \infty$?* The answer depends on whether the process is *ergodically stable* – a concept developed in Section 8.7.

8.1 Preliminaries – What Is a Queueing System? V

Key Takeaway

The goal of steady-state simulation is to estimate $\mathbb{E}[X^*]$ (the stationary mean) by running the system long enough that $\hat{X}(t)$ is a reliable proxy for it. How long is “long enough” is governed by the *time-average variance constant* ς^2 (varsigma squared, also called TAVC), which quantifies how strongly the process is autocorrelated over time.

8.2 Modeling Arrival Streams – Counting Processes I

Many queueing problems require modeling entities arriving at a system randomly. We use *counting processes* to formalize this.

Counting Process $N(t)$

A **counting process** $N(t)$, $t \geq 0$, records the total number of arrivals up to time t (not including time 0). Formally:

- $N(0) = 0$ (no arrivals before time zero).
- $N(t)$ is non-decreasing and right-continuous with unit upward jumps.

The notation $N(s, t] := N(t) - N(s)$ denotes the number of arrivals in the half-open interval $(s, t]$. Equivalently we can describe arrivals through the sequence of **arrival times** $0 < A_1 < A_2 < \dots$, giving:

$$N(t) = \#\{i \geq 1 : A_i \leq t\}, \quad \tau_1 = A_1, \quad \tau_i = A_i - A_{i-1} \quad (i \geq 2).$$

Here τ_i (tau sub i) is the i -th **inter-arrival time**, the waiting time between consecutive arrivals.

8.2 Modeling Arrival Streams – Counting Processes II

The most important special case is the **Poisson process**, which arises when inter-arrival times are independent and exponentially distributed. The exponential distribution $\text{Exp}(\lambda)$ (Exp lambda, with rate $\lambda > 0$) has mean $1/\lambda$ and the *memoryless property*: the remaining time until the next arrival does not depend on how long we have already waited. Here λ (lambda) is the **rate parameter**, measuring arrivals per unit time.

8.2.1 Homogeneous Poisson Process I

The **homogeneous Poisson process** is the most fundamental arrival model. “Homogeneous” means the arrival rate is constant over time.

Definition 8.2.1 – Homogeneous Poisson Process

A counting process $N(t)$ is a **homogeneous Poisson process** with **intensity** $\lambda > 0$ if its inter-arrival times $\tau_1, \tau_2, \dots$ are i.i.d. (independent and identically distributed) random variables with distribution $\text{Exp}(\lambda)$.

Arrivals occur at a constant average rate of λ per unit time, and the time until the next arrival is always exponentially distributed regardless of the history – this is the memoryless property.

Proposition 8.2.2 – Equivalent Characterization

A counting process $N(t)$ is a Poisson process with intensity λ if and only if:

- 1 Numbers of arrivals in *disjoint* time intervals are *independent*.
- 2 For every $t \geq 0$ and $h > 0$: $N(t, t + h] \sim \text{Poi}(\lambda h)$, meaning $N(t, t + h]$ follows a Poisson distribution with mean λh .

8.2.1 Homogeneous Poisson Process II

From this, a useful approximation for small intervals h is:

$$\mathbb{P}(N(t, t + h] = 1) = \lambda h e^{-\lambda h} \approx \lambda h,$$

so the probability of exactly one arrival in a tiny window is approximately λh . This is why λ (lambda) is called the **intensity** or **arrival rate**.

Furthermore, given that exactly n arrivals occurred in $[0, t]$, their positions are i.i.d. uniform on $[0, t]$. This *order-statistics property* leads to a second simulation algorithm.

8.2.1 Homogeneous Poisson Process III

Algorithm 73 – PoiPro(λ): Inter-arrival Method

Simulate a Poisson process on $[0, t_{\max}]$ by generating one inter-arrival time at a time. Here $t_{\max}$ ($t_{\max}$) is the simulation horizon.

- ① Create an empty list A.
- ② Draw $U \sim \mathcal{U}(0, 1)$ (U uniform on 0 to 1); set $t = -\log(U)/\lambda$, giving $\tau_1 \sim \text{Exp}(\lambda)$.
- ③ **While** $t \leq t_{\max}$: append t to A; draw $U \sim \mathcal{U}(0, 1)$; set $t \leftarrow t - \log(U)/\lambda$.
- ④ Return A.

The transformation $-\log(U)/\lambda$ converts a uniform random variable to an exponential, using the inverse-CDF method.

8.2.1 Homogeneous Poisson Process IV

Algorithm 74 – PoiProSorted(λ): Order-Statistics Method

Simulate all arrivals simultaneously, using the uniform distribution.

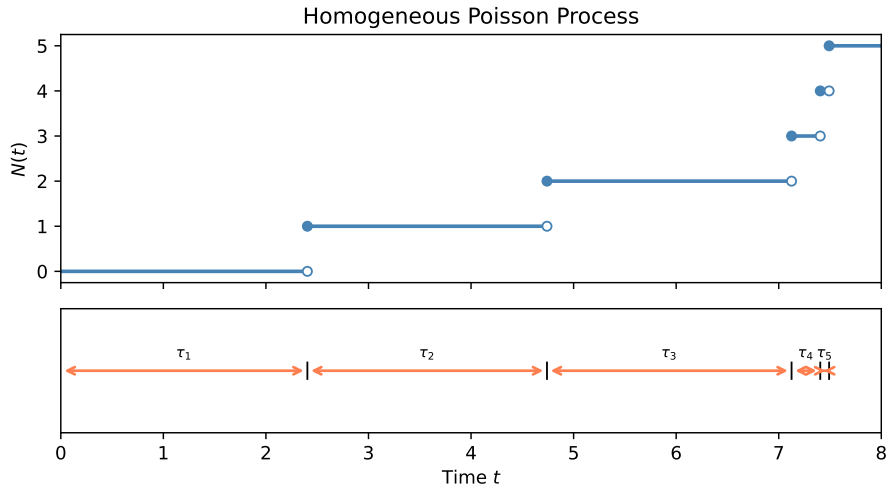
- 1 Sample $K \sim \text{Poi}(\lambda t_{\max})$. Here K is the total number of arrivals, with mean $\lambda t_{\max}$.
- 2 Generate K i.i.d. uniform random variables on $[0, t_{\max}]$.
- 3 Sort and return them as $(A_{(1)} \leq \dots \leq A_{(K)})$.

Both algorithms produce the same distribution. Algorithm 73 is a sequential loop, suitable when we do not know K in advance. Algorithm 74 generates all arrivals at once, which is often faster when $\lambda t_{\max}$ is large.

How to Read the Poisson Process

A larger λ means more frequent arrivals. The number of arrivals in any fixed interval of length h is always Poisson with mean λh , regardless of what happened before – this is independent increments. Real examples: phone calls to a call center, particle emissions in a radiation detector, customer arrivals at a supermarket.

8.2.1 Homogeneous Poisson Process V



Python: Homogeneous Poisson Process (Algorithms 73 & 74)

```
import numpy as np

def simulate_poisson(lam, t_max, rng=None):
    """Algorithm PoiPro: simulate Poisson(lambda) on [0, t_max]."""
    if rng is None:
        rng = np.random.default_rng()
    arrivals = []
    t = -np.log(rng.uniform()) / lam      # first inter-arrival ~ Exp(lam)
    while t <= t_max:
        arrivals.append(t)
        t += -np.log(rng.uniform()) / lam # next inter-arrival
    return np.array(arrivals)

def simulate_poisson_sorted(lam, t_max, rng=None):
    """Algorithm PoiProSorted: simulate via order-statistics."""
    if rng is None:
        rng = np.random.default_rng()
    K = rng.poisson(lam * t_max)          # total number of arrivals
    arrivals = rng.uniform(0, t_max, size=K)
    return np.sort(arrivals)

# Example: lambda=2, simulate on [0,10]; expected ~20 arrivals
rng = np.random.default_rng(42)
A = simulate_poisson(lam=2.0, t_max=10.0, rng=rng)
print(f"Inter-arrival method: {len(A)} arrivals") # expect ~20
B = simulate_poisson_sorted(lam=2.0, t_max=10.0, rng=rng)
print(f"Sorted method: {len(B)} arrivals")
```

8.2.2 Non-homogeneous Poisson Process (NHPP) I

A homogeneous Poisson process has the same arrival rate λ at all times. This is inadequate for many real-world systems: call centers have peak hours, retail stores have rush periods, and network traffic varies throughout the day. A **non-homogeneous Poisson process** (NHPP) generalizes this by allowing the intensity to be a function of time, $\lambda(t)$.

Definition 8.2.3 – Non-homogeneous Poisson Process

A counting process $N(t)$ is an **NHPP** with **intensity function** $\lambda(t) \geq 0$ if:

- 1 Numbers of arrivals in disjoint intervals are independent.
- 2 $N(s, t]$ has a Poisson distribution with mean $\int_s^t \lambda(u) du$ (integral of lambda from s to t).

Here $\lambda(t)$ (lambda of t) is the local arrival rate at time t : the expected number of arrivals in a tiny window $[t, t + dt]$ equals $\lambda(t) dt$. When $\lambda(t) = \lambda$ is constant, the NHPP reduces to the homogeneous case.

8.2.2 Non-homogeneous Poisson Process (NHPP) II

Thinning Algorithm (Lewis-Shedler). The main simulation technique for NHPPs is **thinning**. The idea is: first generate arrivals at the maximum possible rate $\lambda_{\max}$ (an upper bound on $\lambda(t)$), then randomly *discard* each proposed arrival with probability proportional to how much the true rate falls short of the maximum.

Proposition 8.2.4 – Thinning

Let $\lambda_{\max} \geq \lambda(t)$ for all $t \in (0, t_{\max})$. Generate a homogeneous Poisson process at rate $\lambda_{\max}$ to produce candidate arrivals. Retain each candidate arrival at time A_i independently with probability

$$p(A_i) = \frac{\lambda(A_i)}{\lambda_{\max}}.$$

Here $p(A_i)$ (p of A sub i) is the **acceptance probability** at time A_i , and $\lambda_{\max}$ (λ max) is the upper bound on the intensity. The retained process is an NHPP with intensity $\lambda(t)$.

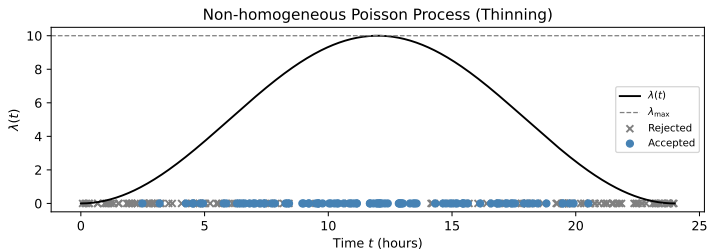
At times when $\lambda(t) = \lambda_{\max}$, every candidate is accepted ($p = 1$); at quiet times where $\lambda(t) = 0.1 \lambda_{\max}$, only 10% are kept.

8.2.2 Non-homogeneous Poisson Process (NHPP) III

Algorithm 75 – NH-Poi-thinning($\lambda(t)$)

Require: $\lambda_{\max} = \max_{t \in [0, t_{\max}]} \lambda(t)$.

- 1 Create empty list A; draw $U \sim \mathcal{U}(0, 1)$; set $t = -\log(U)/\lambda_{\max}$.
- 2 **While** $t \leq t_{\max}$: compute $p = \lambda(t)/\lambda_{\max}$; draw $U_1 \sim \mathcal{U}(0, 1)$; if $U_1 \leq p$, append t to A; draw $U_2 \sim \mathcal{U}(0, 1)$ and set $t \leftarrow t - \log(U_2)/\lambda_{\max}$.
- 3 Return A.



8.2.2 Non-homogeneous Poisson Process (NHPP) IV

Inversion Method (Algorithm 76). An alternative approach uses the *integrated intensity function* $h(t) = \int_0^t \lambda(s) ds$ (h of t , the cumulative intensity up to time t).

Algorithm 76 – NH-Poi($\lambda(t)$): Inversion Method

- 1 Sample $K \sim \text{Poi}(h(t_{\max}))$, where $h(t_{\max}) = \int_0^{t_{\max}} \lambda(s) ds$.
- 2 Compute the c.d.f. $F(s) = h(s)/h(t_{\max})$ and its generalized inverse $F^{\leftarrow}$ (the quantile function).
- 3 Draw $U_1, \dots, U_K$ i.i.d. $\mathcal{U}(0, 1)$; set $A_i = F^{\leftarrow}(U_i)$.
- 4 Return sorted values $(A_{(1)}, \dots, A_{(K)})$.

Algorithm 76 requires computing $F^{\leftarrow}$ analytically or numerically, which is extra work not needed by thinning (Algorithm 75). The thinning approach is therefore more widely used in practice.

8.2.2 Non-homogeneous Poisson Process (NHPP) V

Worked Example: Sinusoidal Call Center

Suppose $\lambda(t) = 10(0.5 + 0.5 \sin(2\pi t/24 - \pi/2))$ models a call center with a 24-hour daily cycle. At midnight ($t = 0$): $\lambda(0) \approx 0$ (no calls). At noon ($t = 12$): $\lambda(12) = 10$ (peak 10 calls/hour). So $\lambda_{\max} = 10$. In thinning: propose arrivals at rate 10, accept each at time t with probability $p(t) = \lambda(t)/10$. Expected total calls per day: $\int_0^{24} \lambda(t) dt = 10 \times 12 = 120$.

Python: NHPP Thinning (Algorithm 75)

```
import numpy as np

def nhpp_thinning(lam_func, lam_max, t_max, rng=None):
    """NH-Poi-thinning: simulate NHPP with intensity lam_func(t).
    lam_func must satisfy lam_func(t) <= lam_max for all t in [0,t_max]."""
    if rng is None:
        rng = np.random.default_rng()
    A = []
    t = -np.log(rng.uniform()) / lam_max    # first candidate ~ Exp(lam_max)
    while t <= t_max:
        p = lam_func(t) / lam_max           # acceptance probability
        if rng.uniform() <= p:              # accept with prob p
            A.append(t)
        t += -np.log(rng.uniform()) / lam_max    # next candidate
    return np.array(A)

# Sinusoidal daily pattern: peaks at noon (t=12), quiet at midnight
lam_func = lambda t: 10 * (0.5 + 0.5 * np.sin(2*np.pi*t/24 - np.pi/2))
rng = np.random.default_rng(7)
A = nhpp_thinning(lam_func, lam_max=10.0, t_max=24.0, rng=rng)
print(f"Arrivals in one day: {len(A)}")    # expected ~120
print(f"First five times (hours): {np.array(A[:5]).round(2)}")
```

8.2.3 Poisson Process on General Spaces I

The Poisson process generalizes naturally to any measurable space $E \subset \mathbb{R}^d$ ($\mathbb{R}$ to the power d , meaning d -dimensional Euclidean space), such as a two-dimensional spatial region (e.g., locations of customers in a city).

Definition 8.2.6 – Poisson Point Process on $E \subset \mathbb{R}^d$

Let Λ (Lambda, capital lambda) be an **intensity measure** on E . A point process N on E is a **Poisson point process** with intensity measure Λ if:

- ① For all disjoint sets $B_1, \dots, B_n \subset E$, the random variables $N(B_1), \dots, N(B_n)$ are independent.
- ② $N(B) \sim \text{Poi}(\Lambda(B))$ for every measurable $B \subset E$.

Here $N(B)$ denotes the number of points falling inside the region B .

Special cases:

- On $E = [0, t_{\max}]$ with $\Lambda(dt) = \lambda dt$: the homogeneous Poisson process.
- With $\Lambda(dt) = \lambda(t) dt$: the NHPP.
- On a rectangle $E \subset \mathbb{R}^2$ with $\Lambda(d\mathbf{x}) = \lambda d\mathbf{x}$: a spatial Poisson process.

8.2.3 Poisson Process on General Spaces II

When $\Lambda(E) < \infty$ (total intensity is finite), simulation is easy:

Algorithm 77 – Poisson Process on E with Finite $\Lambda(E)$

- 1 Generate $T \sim \text{Poi}(\Lambda(E))$. Here T (capital T) is the total number of points; its expected value is $\Lambda(E)$.
- 2 Draw $\xi_1, \dots, \xi_T$ (ξ sub 1 to T) independently from E with probability distribution $\Lambda(d\mathbf{x})/\Lambda(E)$.
- 3 Return the random set $\{\xi_1, \dots, \xi_T\}$.

8.2.3 Poisson Process on General Spaces III

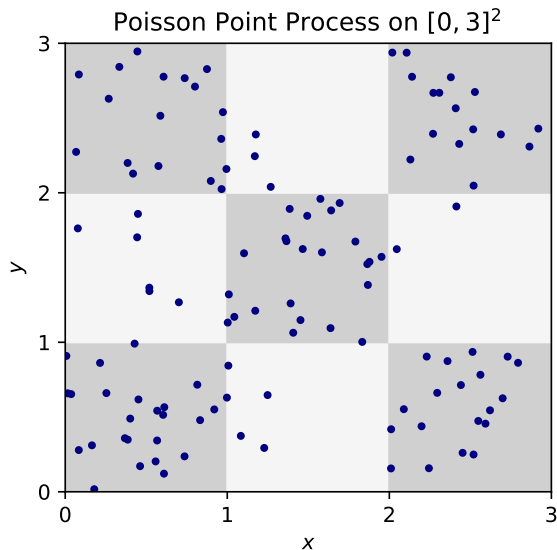


Fig. 8.3: Heterogeneous intensity on $[0, 3]^2$. The domain is partitioned into 9 cells $E_1, \dots, E_9$. Grey (shaded) cells have intensity $\Lambda = 20$ points per cell; light cells have $\Lambda = 5$. Each cell independently generates a Poisson number of uniformly distributed points. Left and right panels are two independent realizations of the same model.

8.3 Birth-and-Death Processes and Markovian Queues I

The simplest queueing models have the Markov property: given the current number of tasks $L(t) = i$ in the system at time t , the future behavior of the queue depends only on i and not on the entire history of arrivals and departures. These models are called **birth-and-death (B&D) processes**.

“Birth” means an arrival (queue length increases by 1); “death” means a departure (queue length decreases by 1).

Birth-and-Death Process

A continuous-time Markov process $X(t)$ on state space $\{0, 1, 2, \dots\}$ is a **birth-and-death process** with birth rates $(\lambda_i)_{i \geq 0}$ (lambda sub i) and death rates $(\mu_i)_{i \geq 1}$ (mu sub i , with $\mu_0 = 0$) if, when in state i :

- A *birth* (jump to state $i + 1$) occurs at rate λ_i .
- A *death* (jump to state $i - 1$) occurs at rate μ_i (only if $i \geq 1$).
- The sojourn time before the next event is $\tau \sim \text{Exp}(\lambda_i + \mu_i)$, i.e., exponential with rate equal to the total event rate.
- Given that an event occurs, it is a birth with probability $\lambda_i / (\lambda_i + \mu_i)$ and a death with probability $\mu_i / (\lambda_i + \mu_i)$.

8.3 Birth-and-Death Processes and Markovian Queues II

Algorithm 78 – BD: Simulate B&D Process on $[0, t_{\max}]$

Require: Parameters $(\lambda_k, \mu_k)_{k \geq 0}$; initial state $y_0 = i$.

- ① Create empty list JS; set $t = 0$.
- ② **While** $t \leq t_{\max}$: append $[t, i]$ to JS; generate $U_1, U_2 \sim \mathcal{U}(0, 1)$; if $U_1 \leq \lambda_i / (\lambda_i + \mu_i)$ then $\xi = +1$, else $\xi = -1$; draw $\tau = -\log(U_2) / (\lambda_i + \mu_i)$; set $t = t + \tau$; set $i = (i + \xi)_+$ (the next state, floored at 0).
- ③ Return JS.

8.3 Birth-and-Death Processes and Markovian Queues III

Kendall Notation: $A/S/c$

Standard queueing systems are described by three symbols:

- **A** = arrival (inter-arrival time) distribution: **M** = Markovian (exponential), **G** = general, **GI** = i.i.d. general, **D** = deterministic.
- **S** = service time distribution: same codes.
- **c** = number of servers; $c = \infty$ means no waiting possible.

The **M/M/1 queue** is the fundamental single-server queue: Poisson arrivals at rate λ (lambda) and exponential service times with rate μ (mu), so mean service time $= 1/\mu$. It is a B&D process with $\lambda_i = \lambda$ and $\mu_i = \mu$ for all valid i .

The **traffic intensity** $\rho = \lambda/\mu$ (rho, Greek letter “rho”) captures stability: $\rho < 1$ for stability, $\rho \geq 1$ for instability.

8.3 Birth-and-Death Processes and Markovian Queues IV

M/M/1 Steady-State Distribution

When $\rho < 1$, the stationary distribution of queue length L^* (L-star) is geometric:

$$\pi_i = \mathbb{P}(L^* = i) = (1 - \rho) \rho^i, \quad i = 0, 1, 2, \dots$$

Here π_i (pi sub i) is the probability of finding exactly i tasks in the system at steady state. The steady-state mean number of tasks:

$$\bar{l} = \mathbb{E}[L^*] = \sum_{i=0}^{\infty} i (1 - \rho) \rho^i = \frac{\rho}{1 - \rho} = \frac{\lambda}{\mu - \lambda}.$$

Here $\bar{l}$ (l-bar) is the long-run average queue length. When $\rho \geq 1$, the queue is unstable and $L(t) \rightarrow \infty$ almost surely.

8.3 Birth-and-Death Processes and Markovian Queues V

Worked Example: M/M/1 Queue Length

Take $\lambda = 0.5$ arrivals per unit time, $\mu = 1$ service per unit time. Then $\rho = 0.5/1 = 0.5$. Mean queue length: $\bar{I} = 0.5/(1 - 0.5) = 1$. The server is idle with probability $\pi_0 = 1 - \rho = 50\%$.

Now take $\lambda = 8/9$, $\mu = 1$, so $\rho = 8/9 \approx 0.889$. Then $\bar{I} = (8/9)/(1/9) = 8$. Going from $\rho = 0.5$ to $\rho = 0.889$ increased the mean queue length 8-fold! This illustrates the extreme sensitivity of queuing systems to traffic intensity near 1.

To estimate $\bar{I}$ from a simulation run on $[0, t_{\max}]$ with jump times $0 = a_0 < a_1 < \dots < a_k \leq t_{\max}$ and state $y_j = L(a_j)$:

$$\hat{I}(t_{\max}) = \frac{1}{t_{\max}} \int_0^{t_{\max}} L(s) ds = \frac{1}{t_{\max}} \sum_{j=0}^k y_j (a_{j+1} - a_j).$$

8.3 Birth-and-Death Processes and Markovian Queues VI

M/M/1 Queue Simulations ($\mu = 1$)

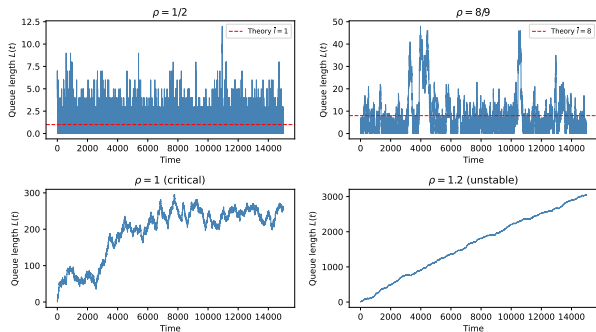


Fig. 8.4: Simulations of $L(t)$ for $\mu = 1$ and four values of λ . **(a)** $\rho = 1/2$: stable, $\bar{L} = 1$, estimate $\hat{L} = 0.98$. **(b)** $\rho = 8/9$: stable, $\bar{L} = 8$, but estimate $\hat{L} = 6.67$ (run too short). **(c)** $\rho = 1$: critical, mean diverges. **(d)** $\rho = 1.2$: unstable, $L(t) \rightarrow \infty$.

Python: Birth-and-Death / M/M/1 Simulation (Algorithm 78)

```
import numpy as np

def simulate_bd(lam_func, mu_func, t_max, y0=0, rng=None):
    """Algorithm 78: Simulate a birth-and-death process on [0, t_max].
    lam_func(i) = birth rate from state i, mu_func(i) = death rate."""
    if rng is None:
        rng = np.random.default_rng()
    t, y = 0.0, y0
    JS = [(0.0, y0)]
    while t < t_max:
        lam_i = lam_func(y)
        mu_i = mu_func(y)
        rate = lam_i + mu_i
        if rate == 0:
            break
        t += rng.exponential(1.0 / rate)
        if t >= t_max:
            break
        u = rng.uniform()
        y = y + 1 if u < lam_i / rate else max(y - 1, 0)
        JS.append((t, y))
    JS.append((t_max, y))
    times = np.array([e[0] for e in JS])
    states = np.array([e[1] for e in JS])
    l_hat = np.sum(states[:-1] * np.diff(times)) / t_max
    return l_hat, JS
```

8.4 Basic Concepts of Queueing Theory I

We now set up general notation for single- and multi-server queues. For the i -th task:

- A_i = arrival time (when the task joins the system).
- S_i = service time (how long the task takes to process, once started).
- W_i = waiting time (time spent in the queue before service starts).
- $D_i = A_i + W_i + S_i$ = departure time (when the task leaves).
- Sojourn time = $W_i + S_i$ (total time from arrival to departure).

The process $L(t) = \sum_{i=1}^{\infty} \mathbf{1}(A_i \leq t < D_i)$ is the number of tasks in the system at time t (including those being served). Here $\mathbf{1}(\cdot)$ is the indicator function, equal to 1 if the condition holds.

We say that arrival times A_i follow a **renewal process** if the inter-arrival times $\tau_i = A_i - A_{i-1}$ are i.i.d. For example, GI/G/1 refers to a single-server system with general i.i.d. inter-arrival times and general i.i.d. service times.

8.4 Basic Concepts of Queueing Theory II

GI/G/c Queueing System

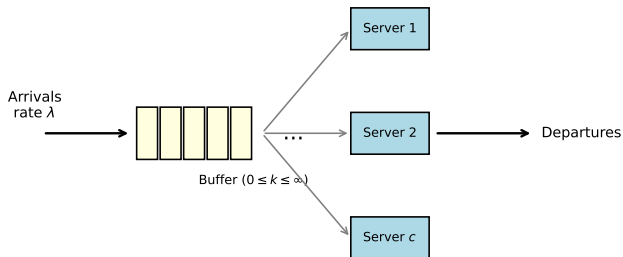


Fig. 8.5 & 8.6: Two standard configurations. Left: one shared queue feeding into c servers (each arriving task joins a single waiting line). Right: c separate queues, one per server. In the shared-queue model, each task goes to the next free server.

8.4 Basic Concepts of Queueing Theory III

Traffic Intensity ρ

The **traffic intensity** (also called the **load** or **utilization**) is

$$\rho = \frac{\mathbb{E}[S_1]}{\mathbb{E}[\tau_1]} = \lambda \mathbb{E}[S_1],$$

where $\mathbb{E}[S_1]$ (E of S sub 1) is the mean service time, $\mathbb{E}[\tau_1]$ is the mean inter-arrival time, and $\lambda = 1/\mathbb{E}[\tau_1]$ (lambda) is the arrival rate. The symbol ρ (varrho, a variant of “rho”) measures how much work arrives per unit time relative to capacity. For a c -server system, stability requires $\rho < c$; with one server, stability requires $\rho < 1$.

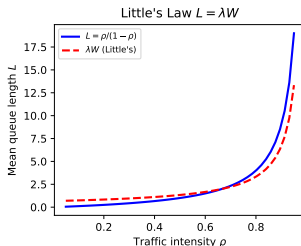
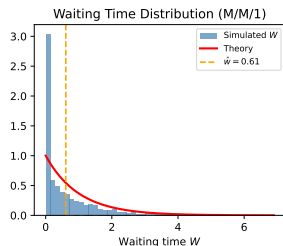
8.4 Basic Concepts of Queueing Theory IV

Little's Law

Under mild conditions, for *any* stable queueing system:

$$\bar{I} = \lambda \bar{w},$$

where $\bar{I}$ (I-bar) is the steady-state mean number of tasks in the system, λ (lambda) is the arrival rate, and $\bar{w}$ (w-bar) is the steady-state mean *sojourn time* (total time from arrival to departure). This connects the three most important performance measures: occupancy ($\bar{I}$), throughput (λ), and latency ($\bar{w}$), and holds regardless of distributions.



Application to M/M/1: We know $\bar{I} = \rho / (1 - \rho)$. Applying Little's Law: $\bar{w} = \bar{I} / \lambda = 1 / (\mu - \lambda)$. With $\lambda = 0.5$, $\mu = 1$: $\bar{I} = 1$ task on average, and $\bar{w} = 2$ time units sojourn per task.

8.4 Basic Concepts of Queueing Theory V

Lindley Recursion for GI/G/1 – Equation (8.4.1)

Under First Come First Served (FCFS) discipline with one server, the waiting times satisfy a simple recursion. Starting with $W_1 = 0$ (first task finds the system empty):

$$W_{i+1} = (W_i + S_i - \tau_{i+1})_+, \quad i = 1, 2, \dots$$

Here $(x)_+ = \max(x, 0)$ is the positive part of x , W_i (W sub i) is the waiting time of task i , S_i (S sub i) is its service time, and τ_{i+1} (τ sub i plus 1) is the inter-arrival time before task $i + 1$.

How to Read the Lindley Recursion

When task $i + 1$ arrives, the server has $W_i + S_i$ total remaining work. Subtracting the inter-arrival time τ_{i+1} gives the remaining workload when task $i + 1$ arrives. If positive, task $i + 1$ waits; if negative, it finds the server idle, so $W_{i+1} = 0$. This recursion is valid for any sample path.

8.4 Basic Concepts of Queueing Theory VI

Pollaczek-Khinchin (PK) Formula for M/G/1

When arrivals are Poisson at rate λ and service times have general distribution with mean $\mathbb{E}[S_1] = 1/\mu$ and $\mathbb{E}[S_1^2] < \infty$:

$$\tilde{w} = \frac{\lambda \mathbb{E}[S_1^2]}{2(1 - \lambda \mathbb{E}[S_1])}.$$

Here $\tilde{w}$ (w-tilde) is the steady-state mean waiting time in queue. $\mathbb{E}[S_1^2]$ (E of S sub 1 squared) captures the variability of service times: more variable service leads to longer waiting even with the same average rate.

8.4 Basic Concepts of Queueing Theory VII

Virtual Waiting Time (Workload) Process – Equation (8.4.4)

The **virtual waiting time** (or *workload*) $V(t)$ is the amount of work the server would face if a new task arrived at time t :

$$V(t) = \sum_{i=1}^{\infty} (W_i + S_i - t)_+ \mathbf{1}(A_i \leq t < A_{i+1}), \quad t \in [0, t_{\max}].$$

It is a piecewise linear process: it decreases at rate 1 between arrivals (as the server processes work) and jumps up by S_i at each arrival time A_i . It is directly computable from the simulated sequence (S_i) , (A_i) , and (W_i) .

8.4 Basic Concepts of Queueing Theory VIII

M/G/ ∞ System: Infinite Servers, No Waiting

In an infinite-server system, each arriving task immediately starts service. With Poisson arrivals (rate λ) and i.i.d. service times $S \sim G$ with mean $\mathbb{E}[S] = 1/\mu$ and load $\rho = \lambda/\mu$:

$$\mathbb{P}(L^* = k) = \frac{\rho^k}{k!} e^{-\rho}, \quad k = 0, 1, \dots$$

The steady-state distribution is Poisson with mean ρ , and crucially this holds for *any* service distribution G – only the mean service time $\mathbb{E}[S]$ matters. Equations (8.4.5)–(8.4.8) confirm that the stationary Poisson distribution also gives: $\lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t L(s) ds = \rho$ a.s.

8.4 Basic Concepts of Queueing Theory IX

Worked Example: GI/G/1 Waiting Times (Table 8.1)

Five tasks: $(\tau_1, S_1) = (1, 2.5)$, $(\tau_2, S_2) = (1.5, 2.5)$, $(\tau_3, S_3) = (4.5, 0.5)$, $(\tau_4, S_4) = (1, 5)$, $(\tau_5, S_5) = (1, 1)$.

Arrival times: $A_1 = 1$, $A_2 = 2.5$, $A_3 = 7$, $A_4 = 8$, $A_5 = 9$. Applying the Lindley recursion with $W_1 = 0$:

$$W_2 = (0 + 2.5 - 1.5)_+ = 1.0 \quad W_3 = (1 + 2.5 - 4.5)_+ = 0.0$$

$$W_4 = (0 + 0.5 - 1.0)_+ = 0.0 \quad W_5 = (0 + 5 - 1.0)_+ = 4.0$$

Departures: $D_1 = 3.5$, $D_2 = 6$, $D_3 = 7.5$, $D_4 = 13$, $D_5 = 14$.

8.5 Discrete-Event Simulations – The General Framework I

Many processes change state only at specific *event times* – arrivals, departures, failures, repairs. Between consecutive events, the system state remains constant. A **discrete-event simulation (DES)** exploits this structure by advancing the simulation clock directly to the next event rather than stepping through time in small fixed increments.

Core Components of a Discrete-Event Simulation

- 1 **Simulation clock** t : the current simulated time.
- 2 **Event list** (EL): a sorted list of pending events, each with a scheduled time and event type. The simulation always processes the event with the smallest time first.
- 3 **State variables**: e.g., n = current number of tasks in the system, queue contents, server status (busy/idle).
- 4 **Output list** JS: records (t, n) pairs after each event for post-processing and estimation.

The DES approach is far more efficient than time-stepping because it skips all the “idle” time between events.

8.5 Discrete-Event Simulations – The General Framework II

Example 8.5.1 – The On-Off Process

The **on-off process** $K(t)$ takes values in $\{0, 1\}$ (off or on), alternating between state 1 (*on*, e.g., a server is active) and state 0 (*off*, idle). *On*-time durations $T_1^{\text{on}}, T_2^{\text{on}}, \dots$ are i.i.d. with distribution F_{on} (mean $\mathbb{E}T^{\text{on}}$); *off*-time durations are i.i.d. with distribution F_{off} (mean $\mathbb{E}T^{\text{off}}$).

The on-off process $K(t)$ is formally defined as:

$$K(t) = \begin{cases} 1 & \text{for } S_{n-1} \leq t < S_n \text{ and } n \text{ odd,} \\ 0 & \text{for } S_{n-1} \leq t < S_n \text{ and } n \text{ even,} \end{cases}$$

where $S_0 = 0$ and $S_n = S_{n-1} + T_k^{\text{on}}$ or T_k^{off} alternately. The steady-state probability of being *on* is:

$$p = \frac{\mathbb{E}T^{\text{on}}}{\mathbb{E}T^{\text{on}} + \mathbb{E}T^{\text{off}}},$$

a consequence of the renewal reward theorem. To estimate p from simulation:

$$\hat{p}(t_{\max}) = \frac{1}{t_{\max}} \int_0^{t_{\max}} K(s) ds.$$

Python: On-Off Discrete-Event Simulation

```
import numpy as np

def simulate_onoff(F_on, F_off, t_max, rng=None):
    """Discrete-event simulation of on-off process.
    F_on / F_off: callables(rng) returning one sample."""
    if rng is None:
        rng = np.random.default_rng()
    t = 0.0; state = 1    # start in ON state
    integral = 0.0        # accumulate integral of K(t)
    while t < t_max:
        duration = F_on(rng) if state == 1 else F_off(rng)
        new_t = min(t + duration, t_max)
        integral += state * (new_t - t)    # K(s)=state on [t, new_t]
        t = new_t
        if t < t_max:
            state = 1 - state            # flip state
    return integral / t_max              # time-averaged estimator p_hat

rng = np.random.default_rng(5)
# Exp on/off: E[T^on]=3, E[T^off]=1, theory p = 3/(3+1) = 0.75
p_hat = simulate_onoff(
    F_on = lambda r: r.exponential(3.0),
    F_off = lambda r: r.exponential(1.0),
    t_max = 50000, rng=rng)
print(f"p_hat = {p_hat:.5f} (theory = 0.75000)")
```

8.5.1 GI/G/1 Queue Simulation (Algorithm 81) I

For a GI/G/1 queue, we can estimate $\tilde{w}$ (the steady-state mean waiting time) by iterating the Lindley recursion to generate $W_1, W_2, \dots$ and computing their sample mean. To also estimate $\bar{I}$ (mean tasks in system), we need the full discrete-event simulation.

8.5.1 GI/G/1 Queue Simulation (Algorithm 81) II

Algorithm 81 – Simulation of a Single-Server Queue

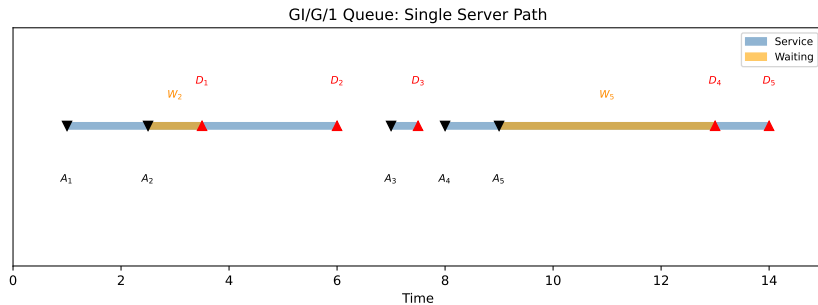
State variables: t_A (time of next scheduled arrival) and t_D (time of next departure; set to $\emptyset$ when server is idle). n = number in system.

- ① Initialize: set $t_A = 0$; generate $\tau \sim F$; set $t_A := \tau$; $EL = [t_A, \emptyset]$; $JS = [[0, 0]]$.
- ② **While** $t_A < t_{\max}$: switch on EL :
 - $EL = [t_A, \emptyset]$ (server idle): $n := 1$; generate $S \sim G$, $t_D = t_A + S$; generate $\tau \sim F$, $t_A := t_A + \tau$; update $EL = [t_A, t_D]$; append $[t_A, n]$ to JS .
 - $t_A < t_D$ (arrival before departure): $n := n + 1$; append to JS ; generate $\tau \sim F$, $t_A := t_A + \tau$.
 - $t_A \geq t_D$ (departure first): $n := n - 1$; append to JS ; if $n > 0$: generate $S \sim G$, $t_D := t_D + S$; else $EL = [t_A, \emptyset]$.
- ③ Flush final departure; append $[t_{\max}, n]$ to JS .

Here F = inter-arrival distribution, G = service distribution.

The estimated mean queue length from JS is the time-weighted average: $\hat{l} = \frac{1}{t_{\max}} \sum_j n_j(t_{j+1} - t_j)$.

8.5.1 GI/G/1 Queue Simulation (Algorithm 81) III



8.5.1 GI/G/1 Queue Simulation (Algorithm 81) IV

Example 8.5.3 – Table 8.1 Data Through Algorithm 81

Five tasks with τ_i and S_i as in Table 8.1 give inter-arrival times (1.0, 1.5, 4.5, 1.0, 1.0) and service times (2.5, 2.5, 0.5, 5, 1). With $t_{\max} = 15$, Algorithm 81 produces the following JS list:

$$[0, 0], [1, 1], [2.5, 2], [3.5, 1], [6, 0], [7, 1], [7.5, 2], [8, 2], \\ [9, 2], [13, 1], [14, 0], [15, 0].$$

From this list one can directly compute the time-averaged queue length and read off waiting times (the orange segments in Fig. 8.9).

8.5.1 GI/G/1 Queue Simulation (Algorithm 81) V

GI/G/c Queue: Workload Vector Recursion (Equation 8.5.2)

For a **GI/G/c queue** (c servers), maintain a sorted vector of server workloads

$\mathbf{V}_i = (V_{i,1} \leq V_{i,2} \leq \dots \leq V_{i,c})$ just before the i -th task arrives. Here $V_{i,j}$ (V sub i comma j) is the remaining work at server j when task i arrives.

The waiting time of task i is $W_i = V_{i,1}$ (the minimum workload; it goes to the least-loaded server). The workload vector updates as:

$$\mathbf{V}_{i+1} = \text{sort}((V_{i,1} + S_i - \tau_{i+1})_+, (V_{i,2} - \tau_{i+1})_+, \dots, (V_{i,c} - \tau_{i+1})_+).$$

All workloads decrease by τ_{i+1} (time passes between arrivals), and task i adds S_i to the minimum workload. For $c = 1$ this reduces to the Lindley recursion. If $\rho = \mathbb{E}[S]/\mathbb{E}[\tau] < c$, the sequence $\mathbf{V}_i$ has a limiting distribution and: $\tilde{w} = \lim_{k \rightarrow \infty} \frac{1}{k} \sum_{i=1}^k V_{i,1}$ a.s.

Python: GI/G/1 Waiting Times via Lindley Recursion

```
import numpy as np

def gig1_waiting_times(n_tasks, F_interarr, G_service, rng=None):
    """Lindley recursion: generate waiting times W_1,...,W_n for GI/G/1."""
    if rng is None:
        rng = np.random.default_rng()
    tau = F_interarr(n_tasks, rng)  # inter-arrival times
    S = G_service(n_tasks, rng)  # service times
    W = np.zeros(n_tasks)
    for i in range(1, n_tasks):
        W[i] = max(W[i-1] + S[i-1] - tau[i], 0.0)
    return W

rng = np.random.default_rng(0)
# M/G/1: Exp(0.5) inter-arrivals, Exp(1) service
# PK formula:  $\tilde{w} = \lambda * E[S^2] / (2 * (1 - \lambda * E[S])) = 0.5 * 2 / (2 * 0.5) = 1.0$ 
W = gig1_waiting_times(
    n_tasks = 100000,
    F_interarr = lambda n, r: r.exponential(2.0, n),  # mean=2=1/lambda
    G_service = lambda n, r: r.exponential(1.0, n),  # mean=1=1/mu
    rng = rng)
print(f"Estimated mean waiting time: {np.mean(W):.4f} (PK theory: 1.0000)")

# M/D/1: deterministic service S=1; PK:  $\tilde{w} = 0.5 * 1 / (2 * (1 - 0.5)) = 0.5$ 
W_det = gig1_waiting_times(
    n_tasks = 100000,
    F_interarr = lambda n, r: r.exponential(2.0, n),
```

8.6 Slotted ALOHA and Random Access Protocols I

Slotted ALOHA is a canonical model of a shared wireless communication channel, proposed in the 1970s. Time is divided into fixed-length slots. Each slot, one or more users may attempt to transmit a packet. If exactly one user transmits, the packet gets through (success). If two or more transmit simultaneously, a *collision* occurs and all packets must be retransmitted.

The three main retransmission strategies:

- **Slotted ALOHA**: after any collision, retransmit in the next slot with a fixed probability h (h , $0 < h < 1$).
- **Exponential Back-off**: retransmit with probability 2^{-i} after i unsuccessful attempts. Here 2^{-i} decreases geometrically in i .
- **Ethernet (Finite Exponential Back-off)**: exponential back-off up to a maximum of K attempts, after which the packet is discarded.

8.6 Slotted ALOHA and Random Access Protocols II

Slotted ALOHA System Model – Equation (8.6.1)

Let X_n (X sub n) be the number of packets *waiting* at the beginning of slot n . New arrivals in slot n : $A_n \sim \text{Poi}(\lambda)$ (Poisson with rate λ), i.i.d. across slots. Retransmission attempts: $Y_n \sim \text{Bin}(X_n, h)$ (Binomial with X_n trials and probability h). Total transmissions in slot n : $Z_n = A_n + Y_n$ (Z sub n). Queue update equation:

$$X_{n+1} = X_n + A_n - \mathbf{1}(Z_n = 1), \quad n = 0, 1, \dots$$

The indicator $\mathbf{1}(Z_n = 1)$ equals 1 only when exactly one packet transmits (a success), removing one packet from the queue.

8.6 Slotted ALOHA and Random Access Protocols III

The **local throughput** at slot n and the **overall throughput**:

$$\gamma_n = \frac{N_n^s}{n}, \quad \gamma = \lim_{n \rightarrow \infty} \gamma_n,$$

where $N_n^s = \sum_{i=1}^n \mathbf{1}(Z_i = 1)$ is the total number of successful transmissions up to slot n .

8.6 Slotted ALOHA and Random Access Protocols IV

Success Probability and Conditional Drift

Given $X_n = k$ waiting packets, the success probability is:

$$\mathbb{P}(Z_n = 1 \mid X_n = k) = e^{-\lambda} \cdot kh(1-h)^{k-1} + \lambda e^{-\lambda}(1-h)^k.$$

The **conditional drift** (expected change in queue length per slot) is:

$$\mathbb{E}[X_{n+1} - X_n \mid X_n = k] = \lambda - e^{-\lambda}(kh(1-h)^{k-1} + \lambda(1-h)^k).$$

As $k \rightarrow \infty$, both terms $kh(1-h)^{k-1} \rightarrow 0$ and $(1-h)^k \rightarrow 0$, so the drift converges to $\lambda > 0$. This positive drift is the key instability result: for large queues, retransmissions dominate and collisions overwhelm the system.

8.6 Slotted ALOHA and Random Access Protocols V

Key Insight: Slotted ALOHA Is Inherently Unstable

For any fixed retransmission probability h and any arrival rate $\lambda > 0$, the chain (X_n) is **transient**: the queue length drifts to infinity with probability 1. The theoretical channel capacity of Slotted ALOHA is $1/e \approx 0.368$ packets per slot (maximum achievable throughput), but even for arrival rates well below this, the system is unstable under a fixed h . This is a classic example of a “metastable” system: it looks stable for many slots, then suddenly jams.

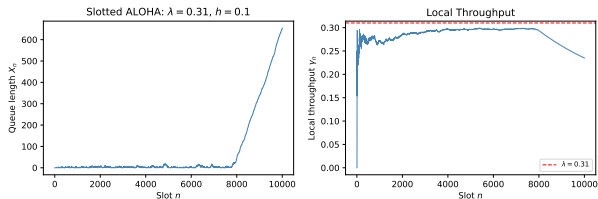


Fig. 8.11: Slotted ALOHA with $\lambda = 0.31$, $h = 0.1$.

(a) Queue length X_n : stable-looking for ~ 5000 slots, then grows linearly.

(b) Local throughput γ_n : initially ≈ 0.31 , then collapses as collisions dominate.

Python: Slotted ALOHA Simulation

```
import numpy as np

def simulate_aloha(lam, h, n_slots, X0=0, rng=None):
    """Simulate Slotted ALOHA with Poisson(lam) arrivals per slot.
    h = retransmission probability. Returns queue lengths and throughputs."""
    if rng is None:
        rng = np.random.default_rng()
    X = X0
    Xs = [X0]; successes = 0; throughputs = [0.0]
    for n in range(1, n_slots + 1):
        A = rng.poisson(lam)                # new arrivals
        Y = rng.binomial(X, h) if X > 0 else 0  # retransmission attempts
        Z = A + Y                          # total transmissions
        success = int(Z == 1)              # 1 iff exactly one
        successes += success
        X = max(X + A - success, 0)
        Xs.append(X)
        throughputs.append(successes / n)
    return np.array(Xs), np.array(throughputs)

# lambda=0.31 is well below capacity 1/e~0.368, yet system is unstable
rng = np.random.default_rng(11)
Xs, gammas = simulate_aloha(lam=0.31, h=0.1, n_slots=10000, rng=rng)
print(f"Final queue length: {Xs[-1]}")
print(f"Final throughput: {gammas[-1]:.4f} (initial rate was ~0.31)")
```

8.7 Ergodically Stable Processes I

To justify using a time average from a single simulation run as an estimator for a steady-state quantity, we need the concept of *ergodic stability*.

Stationary Processes

A stochastic process $(X^*(t))_{t \geq 0}$ (X-star of t) is **stationary** if for all $d \geq 1$ and all time shifts $0 < h_1 < \dots < h_d$, the joint distribution of $(X^*(t + h_1), \dots, X^*(t + h_d))$ does not depend on t .

Key consequences:

- The marginal distribution $\mathbb{P}(X^*(t) \in B)$ is the same for all t ; so $\mathbb{E}[X^*(t)]$ and $\mathbb{Var}[X^*(t)]$ are constant over time.
- The covariance function $R(h) = \text{Cov}(X^*(t), X^*(t + h))$ (R of h) depends only on the *lag* h , not on t .

8.7 Ergodically Stable Processes II

Ergodic Processes – Equation (8.7.2)

A stationary process $(X^*(t))$ is **ergodic** if for any measurable function g with $\mathbb{E}[|g(X^*(h_1), \dots, X^*(h_d))|] < \infty$:

$$\lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t g(X^*(h_1 + s), \dots, X^*(h_d + s)) ds = \mathbb{E}[g(X^*(h_1), \dots, X^*(h_d))], \quad \text{a.s.}$$

In particular, with $g(x) = x$: time averages converge to ensemble averages:

$$\lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t X^*(s) ds = \mathbb{E}[X^*(0)] \quad \text{a.s.}$$

8.7 Ergodically Stable Processes III

Ergodically Stable Processes – Equation (8.7.5)

A stochastic process $(X(t))$ (not necessarily stationary) is **ergodically stable** if there exists a stationary and ergodic process $(X^*(t))$ such that for all suitable functions g :

$$\lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t g(X(h_1 + s), \dots, X(h_d + s)) ds = \mathbb{E}[g(X^*(h_1), \dots, X^*(h_d))], \quad \text{a.s.}$$

Time averages of the actual (non-stationary) process $X(t)$ converge to ensemble averages of the stationary background process $X^*(t)$, regardless of the initial state. In particular (Equation 8.7.1):

$$\hat{X}(t) = \frac{1}{t} \int_0^t X(s) ds \xrightarrow{t \rightarrow \infty} I = \mathbb{E}[X^*(0)] \quad \text{a.s.}$$

Key examples:

8.7 Ergodically Stable Processes IV

- Any irreducible positive recurrent continuous-time Markov chain (CTMC) is ergodically stable. The intensity matrix $\mathbf{Q}$ (Q , whose (i, j) entry is the jump rate from state i to state j) determines the stationary distribution via $\boldsymbol{\pi}\mathbf{Q} = \mathbf{0}$ (π times Q equals the zero vector).
- The on-off process with $\mathbb{E}T^{\text{on}} < \infty$ and $\mathbb{E}T^{\text{off}} < \infty$ is ergodically stable.
- The M/M/1 queue with $\rho < 1$ is ergodically stable.

8.7 Ergodically Stable Processes V

Central Limit Theorem for Ergodic Processes – Equation (8.7.4)

For an ergodic process $(X^*(t))$ satisfying the CLT:

$$\sqrt{t}(\hat{X}^*(t) - \mathbb{E}[X^*(0)]) \xrightarrow{\mathcal{D}} \mathcal{N}(0, \varsigma^2),$$

where ς^2 (varsigma squared) is the **time-average variance constant (TAVC)**, or *asymptotic variance*. Here $\xrightarrow{\mathcal{D}}$ means convergence in distribution (the distribution of the left side approaches that of the right side as $t \rightarrow \infty$), and $\mathcal{N}(0, \varsigma^2)$ is a Gaussian distribution with mean 0 and variance ς^2 .

The fundamental precision-vs-run-length relationship (Equation 8.7.10):

$$b = z_{1-\alpha/2} \frac{\varsigma}{\sqrt{t}},$$

8.7 Ergodically Stable Processes VI

where b (b) is the half-width of the confidence interval, α (alpha) is the significance level (e.g., $\alpha = 0.05$ for 95% confidence), $z_{1-\alpha/2}$ (z sub 1 minus alpha over 2) is the corresponding normal quantile (e.g., $z_{0.975} \approx 1.96$), and t is the simulation run length. To achieve error b with confidence $1 - \alpha$:

$$t \geq \frac{z_{1-\alpha/2}^2 \varsigma^2}{b^2}.$$

How to Read the Run-Length Formula

Longer runs give smaller errors, but with diminishing returns: halving the error requires quadrupling the run length (because $b \propto 1/\sqrt{t}$). A more autocorrelated process (larger ς^2) requires proportionally longer runs. This is the fundamental tradeoff in steady-state simulation.

8.7.1 A Formula for the Asymptotic Variance (TAVC) I

We derive the TAVC ς^2 in terms of the covariance function $R(h) = \text{Cov}(X^*(t), X^*(t+h))$.

Lemma 8.7.5 – TAVC Formula (Equations 8.7.8–8.7.9)

Suppose $\int_0^\infty |R(v)| dv < \infty$ and $\int_0^\infty R(s) ds \neq 0$. Then as $t \rightarrow \infty$:

$$\text{Var}(\hat{X}^*(t)) \sim \frac{2}{t} \int_0^\infty R(v) dv,$$

and hence the TAVC is:

$$\varsigma^2 = 2 \int_0^\infty R(v) dv.$$

Here $R(v)$ (R of v) is the autocovariance at lag v – the covariance between $X^*(t)$ and $X^*(t+v)$ – and the integral sums all these covariances over all positive lags.

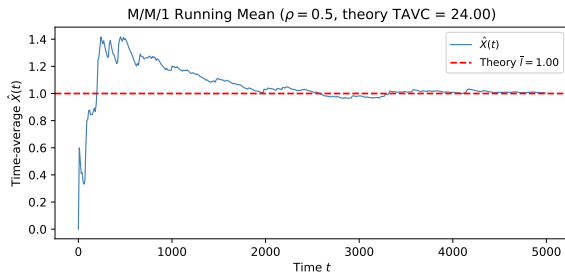
8.7.1 A Formula for the Asymptotic Variance (TAVC) II

Proof sketch: Writing $\text{Var}(\int_0^t X^*(s) ds)$ as a double integral over $(w, v) \in [0, t]^2$, using $\mathbb{E}[X^*(w)X^*(v)] = R(|v - w|)$ (by stationarity with zero mean), one obtains $2 \int_0^t \int_0^v R(v - w) dw dv$, which grows like $2t \int_0^\infty R(u) du$ for large t . Dividing by t^2 gives $\text{Var}(\hat{X}^*(t)) \approx \varsigma^2/t$.

How to Read the TAVC Formula

$\varsigma^2 = 2 \int_0^\infty R(v) dv$ is twice the integral of the autocovariance function. If $R(v) \rightarrow 0$ quickly (the process decorrelates rapidly), ς^2 is small and fewer simulation steps are needed. If $R(v)$ decays slowly (as in heavy-traffic queues), ς^2 is large and much longer runs are required for the same precision.

8.7.1 A Formula for the Asymptotic Variance (TAVC) III



8.7.1 A Formula for the Asymptotic Variance (TAVC) IV

TAVC for the On-Off Process – Kopocinski Formula (Eq. 8.7.11)

For the on-off process with on-times T^{on} and off-times T^{off} (both with finite means), the asymptotic variance of $\hat{p}(t)$ (the estimator of p) is:

$$\varsigma^2 = \frac{(\mathbb{E} T^{\text{on}})^2 \text{Var}[T^{\text{off}}] + (\mathbb{E} T^{\text{off}})^2 \text{Var}[T^{\text{on}}]}{(\mathbb{E} T^{\text{on}} + \mathbb{E} T^{\text{off}})^3}.$$

For exponential on/off times $T^{\text{on}} \sim \text{Exp}(\lambda)$ and $T^{\text{off}} \sim \text{Exp}(\mu)$ (so $\text{Var} = 1/\lambda^2$ and $1/\mu^2$ respectively), using $\rho = \lambda/\mu$ (Equation 8.7.12):

$$\varsigma^2 = \frac{2\lambda\mu}{(\lambda + \mu)^3} = \frac{2\rho}{\mu(1 + \rho)^3}.$$

8.7.1 A Formula for the Asymptotic Variance (TAVC) V

Worked Example: TAVC of Exp/Exp On-Off Process

Take $\lambda = 1/3$ (so $\mathbb{E}T^{\text{on}} = 3$) and $\mu = 1$ (so $\mathbb{E}T^{\text{off}} = 1$). Then $\rho = \mathbb{E}T^{\text{on}}/(\mathbb{E}T^{\text{on}} + \mathbb{E}T^{\text{off}}) = 3/4$ and $\rho = \lambda/\mu = 1/3$.

The TAVC is:

$$\varsigma^2 = \frac{2 \cdot (1/3) \cdot 1}{(1/3 + 1)^3} = \frac{2/3}{(4/3)^3} = \frac{2/3}{64/27} = \frac{9}{32} \approx 0.281.$$

For target absolute error $b = 0.05$ at 95% confidence ($z_{0.975}^2 \approx 4$):

$$t_{\min} = \frac{4 \times 0.281}{0.0025} \approx 450 \text{ time units.}$$

8.7.2 TAVC for Other Systems I

Whitt-Burman Formula (Eq. 8.7.14) for Finite B&D Processes

Let $X(t)$ be a B&D process on $\{0, 1, \dots, n\}$. The stationary distribution satisfies $\pi_i = \pi_0 \frac{\lambda_0 \cdots \lambda_{i-1}}{\mu_1 \cdots \mu_i}$. For $Y(t) = f(X(t))$ with asymptotic mean $I_f = \sum_{i=0}^n f(i)\pi_i$, the TAVC (attributed to D.Y. Burman via Whitt) is:

$$\varsigma_f^2 = 2 \sum_{j=0}^{n-1} \frac{1}{\lambda_j \pi_j} \left[\sum_{i=0}^j (f(i) - I_f) \pi_i \right]^2.$$

Here λ_j (lambda sub j) is the birth rate from state j , π_j (pi sub j) is the stationary probability of state j , and $f(i)$ is the function of interest (e.g., $f(i) = i$ for queue length).

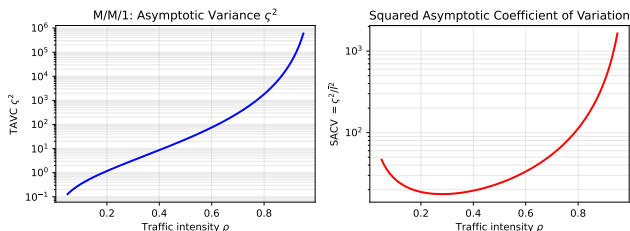
M/M/1 Queue TAVC. For the M/M/1 queue ($\lambda_i = \lambda$, $\mu_i = \mu$), the covariance function of $L(t)$ was derived by Reynolds using a spectral method. The result is:

$$\varsigma_L^2 = \frac{2\rho(1+\rho)}{\mu(1-\rho)^4}, \quad \rho = \lambda/\mu.$$

8.7.2 TAVC for Other Systems II

This formula shows that ζ_L^2 grows as $(1 - \rho)^{-4}$ as $\rho \nearrow 1$: near-critical queues require dramatically longer simulation runs.

M/M/ n Queue. For n servers with $\lambda_i = \lambda$ and $\mu_i = \min(i, n)\mu$, the stationary distribution is the *Erlang-C* distribution and the TAVC can be computed from the Whitt-Burman formula.



8.8 Regenerative Processes I

Regenerative processes are a broad class covering most continuous-time queueing models. Their key feature is that they contain random renewal times at which the process probabilistically “starts over”, enabling cycle-based estimation.

Definition 8.8.1 – Regenerative Process

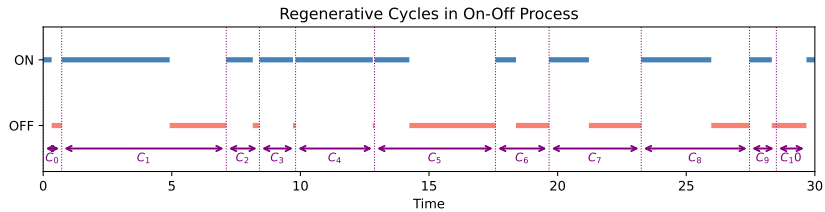
A stochastic process $(X(t), t \geq 0)$ is **regenerative** if there exist random times $0 = T_0 \leq T_1 < T_2 < \dots$ (regeneration times) such that the pairs $(Z_i, C_i)_{i \geq 0}$ are independent, where:

$$C_0 = T_1, \quad C_i = T_{i+1} - T_i \quad (i \geq 1) \quad (\text{cycle lengths}),$$

$$Z_i = \int_{T_i}^{T_{i+1}} X(s) ds \quad (i \geq 1) \quad (\text{integrals over cycles}),$$

and $(Z_i, C_i)_{i \geq 1}$ are i.i.d. The process *restarts probabilistically* at each T_i .

8.8 Regenerative Processes II



8.8 Regenerative Processes III

Examples of regenerative processes:

- **On-off process:** regenerates each time the process transitions to state 1 (on). One cycle $= T^{\text{on}} + T^{\text{off}}$.
- **M/M/1 queue:** $L(t)$ regenerates at times T_i where the queue empties and a new arrival occurs. One cycle = one idle + one busy period.
- **GI/G/1 queue:** regenerates at the start of each busy period (when the queue goes from 0 to 1 customer). The virtual waiting time process $V(t)$ (defined in Eq. 8.4.4) is also regenerative, provided $\mathbb{E}S < \mathbb{E}T$ (i.e., $\rho < 1$).
- **M/G/ ∞ queue:** regenerates when the system is empty and a new arrival occurs.
- Any irreducible positive recurrent Markov chain is regenerative with regeneration at return visits to any fixed state.

8.8 Regenerative Processes IV

Non-delayed regenerative processes. When $T_1 = 0$ (the process starts at a regeneration point), it is called **non-delayed** and denoted $(X^o(t))$. The cycles $(X^o(s), T_i^o \leq s < T_{i+1}^o)$, $i = 1, 2, \dots$ are i.i.d. For the on-off process starting in the on-state, the stationary background process $K^*(t)$ has initial distribution given by:

$$\mathbb{P}(K^*(0) = 1, A^*(0) > x) = \frac{\mathbb{E} T_{\text{on}}}{\mathbb{E} T_{\text{on}} + \mathbb{E} T_{\text{off}}} \cdot \frac{1}{\mathbb{E} T_{\text{on}}} \mathbb{E}(T_{\text{on}} - x)_+.$$

8.8 Regenerative Processes V

Proposition 8.8.2 – Renewal-Reward Theorem (Equation 8.8.1)

If $\mathbb{E}[C_1] < \infty$ and $\mathbb{E}[|Z_1|] < \infty$, then with probability 1:

$$\hat{X}(t) = \frac{1}{t} \int_0^t X(s) ds \rightarrow I = \frac{\mathbb{E}[Z_1]}{\mathbb{E}[C_1]}.$$

The steady-state mean equals the *mean reward per cycle* divided by the *mean cycle length*. If additionally $\mathbb{E}[C_1^2] < \infty$ and $\text{Var}[Z_1] < \infty$, a CLT holds (Equation 8.8.2):

$$\sqrt{t}(\hat{X}(t) - I) \xrightarrow{\mathcal{D}} \mathcal{N}(0, \varsigma_A^2),$$

where $\varsigma_A^2 = \mathbb{E}[\zeta_1^2]/\mathbb{E}[C_1]$ and $\zeta_k = Z_k - I C_k$ (zeta sub k, the centered cycle contribution).

The renewal-reward theorem motivates the **regenerative ratio estimator**. After simulating k complete regeneration cycles (discarding cycle 0):

8.8 Regenerative Processes VI

Regenerative Ratio Estimator – Equations (8.8.6)–(8.8.8)

$$\hat{I}_k = \frac{Z_1 + \cdots + Z_k}{C_1 + \cdots + C_k}.$$

By the LLN, $\hat{I}_k \rightarrow I$ a.s. Under the conditions of Prop. 8.8.2 (Proposition 8.8.6):

$\sqrt{k}(\hat{I}_k - I) \xrightarrow{\mathcal{D}} \mathcal{N}(0, \varsigma_B^2)$, where $\varsigma_B^2 = \mathbb{E}[\zeta_1^2]/(\mathbb{E}[C_1])^2$.

The $1 - \alpha$ confidence interval using k cycles:

$$\hat{I}_k \pm \frac{z_{1-\alpha/2} \hat{\varsigma}_B}{\sqrt{k}},$$

where $\hat{\varsigma}_B^2 = \hat{\varsigma}_A^2 / \hat{C}^2$ with $\hat{C} = \frac{1}{k} \sum_{j=1}^k C_j$ (sample mean cycle length) and $\hat{\varsigma}_A^2 = \frac{1}{k-1} \sum_{j=1}^k (\zeta_j - \bar{\zeta})^2$ with $\zeta_j = Z_j - \hat{I}_k C_j$.

8.8 Regenerative Processes VII

Comparing $\hat{X}(t)$ and $\hat{I}_k$: Both have the same asymptotic confidence interval width (the CIs are identical in length to leading order). The key advantage of $\hat{X}(t)$ is simplicity: no need to identify regeneration times. The key advantage of $\hat{I}_k$ is unbiasedness (once the first cycle is discarded) and that it is directly constructible from cycle data.

8.8 Simulation Results for On-Off Process (Example 8.8.7) I

We illustrate both estimators on the on-off process with true $p = 3/4$:

- **Option 1 (Exp/Exp):** $T^{\text{on}} \sim \text{Exp}(1/3)$, $T^{\text{off}} \sim \text{Exp}(1)$; exponential tails.
- **Option 2 (Par/Gamma):** $T^{\text{on}} \sim \text{Par}(2.5, 1.8)$ (Pareto), $T^{\text{off}} \sim \text{Gamma}(2, 1/2)$; heavier-tailed.

In both cases $\mathbb{E}T^{\text{on}} = 3$ and $\mathbb{E}T^{\text{off}} = 1$, giving $p = 3/(3 + 1) = 0.75$. Both estimators computed from the same simulation run for $t \in [0, 1000]$.

Quantity	Option 1 (Exp/Exp)	Option 2 (Par/Gamma)
True p	0.75000	0.75000
Cycles used (k)	257	263
$\hat{X}(t_{\max})$	0.72733	0.73932
$\hat{I}_k$	0.72472	0.73937
95% CI for $\hat{X}(t)$	(0.656, 0.798)	(0.701, 0.777)
95% CI for $\hat{I}_k$	(0.688, 0.761)	(0.720, 0.759)

8.8 Simulation Results for On-Off Process (Example 8.8.7) II

Both estimators bracket the true value $p = 0.75$. For Option 2 (Pareto/Gamma), the confidence intervals are narrower: since $\text{Par}(2.5)$ has $\alpha > 2$, the variance $\mathbb{E}[S^2] < \infty$, leading to faster convergence. Both estimators are implemented in `ch8_on_off.py`.

Python: Regenerative Estimation for On-Off Process

```
import numpy as np

def regenerative_onoff(F_on, F_off, n_cycles, rng=None):
    """Regenerative ratio estimator for on-off process.
    Cycles begin at each entry into the on state.
    First cycle is discarded for unbiasedness."""
    if rng is None:
        rng = np.random.default_rng()
    Zs, Cs = [], []
    for _ in range(n_cycles + 1):    # +1 to discard first cycle
        T_on = F_on(rng)
        T_off = F_off(rng)
        C = T_on + T_off    # cycle length = on + off
        Z = T_on            # integral of K(t)=1 over cycle = on-length
        Zs.append(Z); Cs.append(C)
    Zs, Cs = np.array(Zs[1:]), np.array(Cs[1:])    # discard first
    I_hat = np.sum(Zs) / np.sum(Cs)               # ratio estimator
    # Estimate TAVC:  $\zeta_j = Z_j - I_{\text{hat}} * C_j$ 
    zeta = Zs - I_hat * Cs
    C_bar = np.mean(Cs)
    s2_B = np.var(zeta, ddof=1) / (C_bar ** 2)
    k = len(Zs)
    ci_half = 1.96 * np.sqrt(s2_B / k)
    return I_hat, ci_half
```

```
rng = np.random.default_rng(3)
```

```
I_hat, ci = regenerative_onoff(
```

8.9 Error Analysis of M/M/1 Queue I

We apply the TAVC framework to the M/M/1 queue to determine how long to simulate for a given precision requirement.

The queue length $L(t)$ is a regenerative ergodically-stable process. Regeneration times are moments when the queue empties and a new arrival occurs:

$$T_1 = \inf\{t > 0 : L(t^-) = 0, L(t) = 1\}, \quad T_i = \inf\{t > T_{i-1} : L(t^-) = 0, L(t) = 1\}.$$

Each cycle = one exponential idle period (duration $\text{Exp}(\lambda)$) plus one busy period. The mean busy period is $1/(\mu - \lambda)$, and its variance is $\text{Var}[\text{Busy}] = (1 + \rho)/[\mu^2(1 - \rho)^3]$.

M/M/1 Steady-State Quantities ($\rho = \lambda/\mu < 1$)

- Steady-state mean queue length: $\bar{l} = \mathbb{E}[L^*] = \frac{\rho}{1 - \rho}$ (Equation 8.9.2).
- Steady-state variance: $\sigma_L^2 = \text{Var}[L^*] = \frac{\rho}{(1 - \rho)^2}$.
- TAVC (Equation 8.9.3): $\varsigma_L^2 = \frac{2\rho(1 + \rho)}{\mu(1 - \rho)^4}$.

8.9 Error Analysis of M/M/1 Queue II

The TAVC ς_L^2 grows as $(1 - \rho)^{-4}$ as $\rho \nearrow 1$. This means that in heavy traffic, the required simulation run length grows as the *fourth power* of $1/(1 - \rho)$.

8.9 Error Analysis of M/M/1 Queue III

The SACV (Squared Asymptotic Coefficient of Variation) is:

$$\text{SACV} = \frac{\varsigma_L^2}{\bar{l}^2} = \frac{2(1+\rho)}{\mu\rho(1-\rho)^2}.$$

Setting a target relative error of 10% ($b = 0.1\bar{l}$) at 95% confidence ($z_{0.975}^2 \approx 4$):

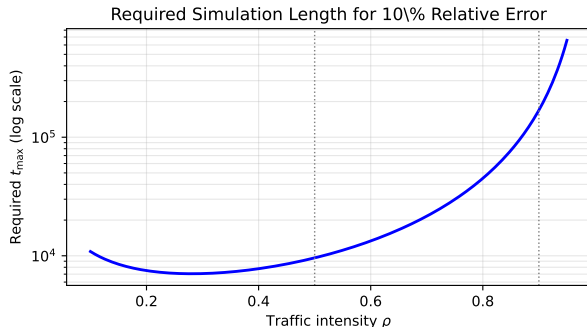
$$t_{\max} = 4 \cdot \frac{\varsigma_L^2}{(0.1\bar{l})^2} = 400 \cdot \text{SACV}.$$

Worked Example 8.9.1 (with $\mu = 1$)

Case (a): $\rho = 1/2$. $\bar{l} = 1$. $\varsigma_L^2 = 2(1/2)(3/2)/[1 \cdot (1/2)^4] = (3/2)/(1/16) = 24$. Required:
 $t_{\max} = 400 \cdot 24/1 = 9600$. Simulation on $[0, 10^4]$ gave $\hat{l} = 0.9822$ (1.78% error – within 10%).

Case (b): $\rho = 8/9$. $\bar{l} = 8$. $\varsigma_L^2 = 2(8/9)(17/9)/[1 \cdot (1/9)^4] \approx 22032$. Required:
 $t_{\max} = 400 \cdot 22032/64 \approx 1.377 \times 10^5$. Simulation on $[0, 10^4]$ gave $\hat{l} = 6.67$ (16.6% error). Since $10^4 \ll 1.377 \times 10^5$, this large error is entirely expected.

8.9 Error Analysis of M/M/1 Queue IV



Heavy traffic scaling: As $\rho \nearrow 1$:

- For *absolute* error b : $t_{\max} = O((1 - \rho)^{-4})$.
From $\rho = 0.5$ to $\rho = 0.95$: factor $100^2 = 10\,000$ more!
- For *relative* error: $t_{\max} = O((1 - \rho)^{-2})$.
From $\rho = 0.5$ to $\rho = 0.95$: factor 100 more.

8.9 Error Analysis of M/G/ ∞ Systems I

For the M/G/ ∞ queue (infinite servers, Poisson arrivals at rate λ , i.i.d. service times $S \sim G$ with mean $\mathbb{E}[S] = \mu^{-1}$), the covariance function can be computed explicitly.

8.9 Error Analysis of M/G/ ∞ Systems II

M/G/ ∞ Covariance Function and TAVC – Equation (8.9.4)

The covariance function of the stationary process $L^*(t)$ (Rolski et al.):

$$R(t) = \lambda \int_t^\infty (1 - G(v)) dv, \quad t \geq 0.$$

Here $G(v) = \mathbb{P}(S \leq v)$ is the c.d.f. of the service time (G of v , the probability that service completes within time v) and $(1 - G(v))$ is the probability that service time exceeds v . The TAVC:

$$\varsigma^2 = 2 \int_0^\infty R(t) dt = \lambda \mathbb{E}[S_1^2].$$

This beautiful result says the TAVC equals λ times the second moment of the service time. For exponential service ($\mathbb{E}[S^2] = 2/\mu^2$): $\varsigma^2 = 2\lambda/\mu^2 = 2\rho$.

The behavior depends critically on the tail of G . For Pareto service $S = cX$, $X \sim \text{Par}(\alpha)$ ($\mathbb{P}(X > x) = (1 + x)^{-\alpha}$):

8.9 Error Analysis of M/G/ ∞ Systems III

- $\alpha > 3$: $\mathbb{E}[S^2] < \infty$, $\varsigma^2 < \infty$, CLT holds. Convergence at the normal $O(1/\sqrt{t})$ rate.
- $2 < \alpha \leq 3$: $\mathbb{E}[S^2] = \infty$, $\varsigma^2 = \infty$. The CLT for $\hat{L}(t)$ breaks down; a non-standard stable limit theorem applies.
- $1 < \alpha \leq 2$: $\mathbb{E}[S] < \infty$ but $\mathbb{E}[S^2] = \infty$. The estimator $\hat{L}(t)$ converges to ϱ (consistent) but extremely slowly.

8.9 Error Analysis of M/G/∞ Systems IV

Algorithm 82 – Simulation of M/G/∞ Systems

Each task i arrives at time A_i (generated as a Poisson process) and departs at $D_i = A_i + S_i$. The integral $\int_0^{t_{\max}} L(s) ds$ equals $\sum_i [\min(t_{\max}, A_i + S_i) - A_i]$ (Equation 8.9.5), because each task i contributes the portion of its sojourn before $t_{\max}$.

- 1 Initialize $t_{\max}$, λ ; set $L = 0$, $T = 0$.
- 2 **While** arrival times $A_i \leq t_{\max}$: generate $A_i \sim \text{Exp}(\lambda)$; generate $S_i \sim G$. If $A_i + S_i \leq t_{\max}$: $L \leftarrow L + S_i$; else: $L \leftarrow L + (t_{\max} - A_i)$.
- 3 Compute $\hat{I} = L/t_{\max}$; return $\hat{I}$.

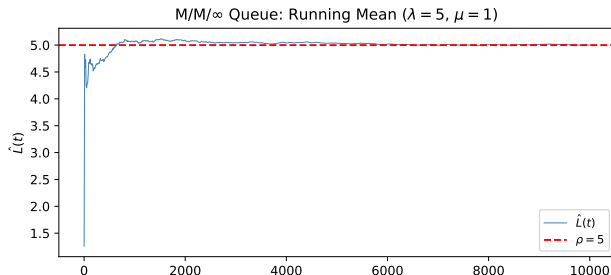


Fig. 8.14: $\hat{L}(t)$ for M/G/∞ with $\lambda = 5$, $\bar{I} = 5$.

(a) $S \sim \text{Exp}(1)$: $\zeta^2 = 10$; converges quickly.

(b) $S = 2.2X$, $X \sim \text{Par}(3.2)$: $\zeta^2 = 18.3$; needs $t > 7320$.

(c) $S = 1.2X$, $X \sim \text{Par}(2.2)$: $\zeta^2 = 60$; needs $t > 24000$.

(d) $S = 0.2X$, $X \sim \text{Par}(1.2)$: $\zeta^2 = \infty$; CLT

Python: M/G/ ∞ Simulation (Algorithm 82)

```
import numpy as np

def simulate_mginf(lam, G_sample, t_max, rng=None):
    """Algorithm 82: Simulate M/G/inf and estimate steady-state mean.
    Uses:  $\hat{l} = \sum_i [\min(t_{\max}, A_i + S_i) - A_i] / t_{\max}$ """
    if rng is None:
        rng = np.random.default_rng()
    arrivals = []
    t = rng.exponential(1.0 / lam)
    while t < t_max:
        arrivals.append(t)
        t += rng.exponential(1.0 / lam)
    arrivals = np.array(arrivals)
    if len(arrivals) == 0:
        return 0.0
    service = G_sample(len(arrivals), rng)
    departures = arrivals + service
    # Each task i contributes  $\min(t_{\max}, D_i) - A_i$  to the integral
    contribution = np.minimum(departures, t_max) - arrivals
    return np.sum(contribution) / t_max

rng = np.random.default_rng(99)
# Exponential service: theory  $\rho = \lambda/\mu = 5/1 = 5$ 
l_hat = simulate_mginf(5.0, lambda n, r: r.exponential(1.0, n), 1e4, rng)
print(f"M/M/inf (Exp, sigma^2=10):      l_hat={l_hat:.4f} (theory=5.0)")

# Pareto(3.2) service, scaled to mean 1
```

Chapter 8 Summary I

Arrival modeling:

- Homogeneous Poisson: i.i.d. $\text{Exp}(\lambda)$ inter-arrivals; simulate via sequential draws (Alg. 73) or order statistics (Alg. 74).
- Non-homogeneous Poisson: time-varying rate $\lambda(t)$; use thinning (Alg. 75) or inversion (Alg. 76).
- General spaces: Alg. 77 draws $K \sim \text{Poi}(\Lambda(E))$ points from the normalized intensity measure.

Queue models:

- M/M/1, M/M/c: B&D processes; stable iff $\rho < 1$ (or $\rho < c$); stationary mean $\bar{I} = \rho/(1 - \rho)$; Little's Law: $\bar{w} = \bar{I}/\lambda$.
- GI/G/1: Lindley recursion
 $W_{i+1} = (W_i + S_i - \tau_{i+1})^+$

Output analysis:

- Time average $\hat{X}(t) \rightarrow I = \mathbb{E}[X^*]$ for ergodically stable processes (Eq. 8.7.1).
- TAVC: $\varsigma^2 = 2 \int_0^\infty R(v) dv$ (Lemma 8.7.5).
- Run length: $t_{\max} = z_{1-\alpha/2}^2 \varsigma^2 / b^2$.
- M/M/1 TAVC: $\varsigma_L^2 = 2\rho(1 + \rho)/[\mu(1 - \rho)^4]$.
- M/G/ ∞ TAVC: $\varsigma^2 = \lambda \mathbb{E}[S^2]$.
- Heavy traffic: required run grows as $(1 - \rho)^{-4}$ for absolute error.

Regenerative estimation:

- Ratio estimator $\hat{I}_k = \sum Z_j / \sum C_j$.
- Same asymptotic CI width as $\hat{X}(t)$.
- Unbiased once first cycle discarded.
- TAVC estimated from cycles via $\varsigma_j^2 = Z_j - \hat{I}_k C_j$.

Chapter 8 Summary II

Summary of Chapter 8 Algorithms:

- **Alg. 73** $\text{PoiPro}(\lambda)$: homogeneous Poisson via sequential inter-arrival times.
- **Alg. 74** $\text{PoiProSorted}(\lambda)$: Poisson via order statistics.
- **Alg. 75** $\text{NH-Poi-thinning}(\lambda(t))$: NHPP via thinning (Lewis-Shedler).
- **Alg. 76** $\text{NH-Poi}(\lambda(t))$: NHPP via inversion.
- **Alg. 77** Poisson process on general space E with finite $\Lambda(E)$.
- **Alg. 78** BD: birth-and-death process simulation.
- **Alg. 79** Repairman problem: DES with failure/repair events and priority queue.
- **Algs. 80–81** DES of GI/G/1 queue using event list.
- **Alg. 82** M/G/ ∞ simulation via arrival-departure pairs.